Университет ИТМО
Факультет программной инженерии и компьютерной техники

Группа: P3311
Студент: Горошников Т.И.
Преподаватель: Потемин И.С.

Отчет по лабораторной работе №2 “Рассчет интеграла функции на заданном промежутке методом Монте-Карло“

Цель работы

Изучить базовые методы Монте-Карло, используемые для интегрирования функций на заданном интервале, и оценить точность интегрирования и основные преимущества и недостатки этих методов.

Используемые формулы

1. Простое интегрирование методом Монте-Карло:

$$I = \frac{b-a}{n} \sum_{i=0}^{n} f(x_i)$$

2. Интегрирование методом Монте-Карло со стратификацией:
   Пусть $j = \overline{1, m}$ — номер отрезка разбиения, l — длина отрезка разбиения.
   Тогда:

$$I = \sum_{j=1}^{m} \frac{l}{n} \sum_{i=0}^{n} f(x_{j_i})$$

3. Интегрирование методом Монте-Карло с выборкой по значимости:

$$I = \frac{1}{n} \sum_{i=0}^{n} \frac{f(x_i)}{p(x_i)}$$

4. Интегрирование методом Монте-Карло с многократной выборкой по значимости:

$$I = \frac{1}{n} \sum_{i=0}^{n} f(x_i) \left(\frac{p_1(x_i)}{w_1(x_i)} + \frac{p_2(x_i)}{w_2(x_i)} \right)$$

5. Интегрирование методом Монте-Карло с использованием русской рулетки:

$$I = \frac{1}{n} \sum_{i=0}^{n} \frac{f(x_i)}{p(x_i) \cdot R}$$

Программа для рассчета интегралов

Весь код можно также найти по ссылке.
Фрагмент файла `main.py` с основными вычислениями:

```
rng = np.random.default_rng(35)
```

```
def calculate_simple_monte_carlo_integral(a, b, function, n):

    samples = rng.uniform(a, b, size=n)

    y = function(samples)

    return np.mean(y) * (b - a)



def calculate_stratified_monte_carlo_integral(a, b, function, step, n):

    partition = np.arange(a, b + 1e-6, step)

    result = 0

    for i in range(len(partition) - 1):

        samples = rng.uniform(partition[i], partition[i + 1], size=n)

        result += np.mean(function(samples)) * (partition[i + 1] -
partition[i])

    return result



def calculate_importance_monte_carlo_integral(a, b, function, pdf,
inv_cdf, n):

    samples = inv_cdf(rng.uniform(0, 1, size=n)) * (b - a) + a

    y = function(samples) / pdf(samples)

    return np.mean(y)



def calculate_multi_importance_monte_carlo_integral(a, b, function,
pdfs, inv_cdfs, weight_functions, n):

    base_samples = rng.uniform(0, 1, size=n)
```

```
    result = 0

    for pdf, inv_cdf, weight_function in zip(pdfs, inv_cdfs,
weight_functions):

        samples = inv_cdf(base_samples) * (b - a) + a

        result += function(samples) / pdf(samples) *
weight_function(samples, pdfs[0], pdfs[1])

    return np.mean(result)



def calculate_russian_roulette_monte_carlo_integral(a, b, function, pdf,
inv_cdf, r, n):

    y = 0

    for _ in range(n):

        sample = inv_cdf(rng.uniform(0, 1)) * (b - a) + a

        if rng.uniform(0, 1) < r:

            y += function(sample) / (pdf(sample) * r)

    return y / n




square = lambda x: x ** 2

square_integral = lambda x: x ** 3 / 3

p1 = lambda x: 2 * (x - 2) / 9

p2 = lambda x: (x - 2) ** 2 / 9

p3 = lambda x: 4 * (x - 2) ** 3 / 81
```

```
f1 = lambda x: x ** (1 / 2)

f2 = lambda x: x ** (1 / 3)

f3 = lambda x: x ** (1 / 4)


w1 = lambda x, pdf1, pdf2: pdf1(x) / (pdf1(x) + pdf2(x))

w2 = lambda x, pdf1, pdf2: pdf2(x) / (pdf1(x) + pdf2(x))

w3 = lambda x, pdf1, pdf2: pdf1(x) ** 2 / (pdf1(x) ** 2 + pdf2(x) ** 2)

w4 = lambda x, pdf1, pdf2: pdf2(x) ** 2 / (pdf1(x) ** 2 + pdf2(x) ** 2)


r1 = 0.5

r2 = 0.75

r3 = 0.95


n1 = 100

n2 = 1000

n3 = 10000

n4 = 100000


a, b = 2, 5


true_integral_value = square_integral(5) - square_integral(2)
```

```
def run_experiments(a, b, true_integral_value):
    results = []

    for n in [n1, n2, n3, n4]:
        estimate = calculate_simple_monte_carlo_integral(a, b, square,
n)
        results.append(
            {
                "method": "simple",
                "n": n,
                "params": "-",
                "estimate": estimate,
                "abs_error": abs(estimate - true_integral_value),
            }
        )

        for step in [1, 0.5]:
            estimate = calculate_stratified_monte_carlo_integral(a, b,
square, step, n)
            results.append(
                {
```

```
                "method": "stratified",
                "n": n,
                "params": f"step={step}",
                "estimate": estimate,
                "abs_error": abs(estimate - true_integral_value),
            }
        )


    for pdf_name, pdf, inv_cdf in zip(["p1", "p2", "p3"], [p1, p2, p3], [f1, f2, f3]):
        estimate = calculate_importance_monte_carlo_integral(a, b, square, pdf, inv_cdf, n)
        results.append(
            {
                "method": "importance",
                "n": n,
                "params": f"pdf={pdf_name}",
                "estimate": estimate,
                "abs_error": abs(estimate - true_integral_value),
            }
        )
```

```
            for r in [r1, r2, r3]:

                estimate =
calculate_russian_roulette_monte_carlo_integral(a, b, square, pdf,
inv_cdf, r, n)

                results.append(

                    {

                        "method": "russian_roulette",

                        "n": n,

                        "params": f"pdf={pdf_name}, r={r}",

                        "estimate": estimate,

                        "abs_error": abs(estimate -
true_integral_value),

                    }

                )



        for weight_name, weight_function_pair in [("balance", [w1, w2]),
("power", [w3, w4])]:

            estimate = calculate_multi_importance_monte_carlo_integral(

                a, b, square, [p1, p3], [f1, f3], weight_function_pair,
n

            )

            results.append(

                {

                    "method": "multi_importance",
```

```
                    "n": n,

                    "params": f"pdfs=p1,p3; weights={weight_name}",

                    "estimate": estimate,

                    "abs_error": abs(estimate - true_integral_value),

                }

            )



    return results
```

Результаты вычислений

Истинное значение интеграла:

$$I = \int_2^5 x^2 dx = \frac{x^3}{3}|_2^5 = \frac{125-8}{3} = 39$$

n	params	estimate	abs_error
100	-	37.7766	1.22343
1000	-	37.9526	1.0474
10000	-	38.9943	0.005676
100000	-	38.9796	0.020392

Результаты простого интегрирования методом Монте-Карло

n	params	estimate	abs_error
100	step=0.5	38.9305	0.069517
100	step=1	38.8105	0.189512
1000	step=0.5	39.0267	0.026703
1000	step=1	38.8825	0.117538
10000	step=0.5	39.0037	0.003674
10000	step=1	39.0022	0.002176
100000	step=0.5	38.9913	0.008701
100000	step=1	39.0109	0.010925

Результаты интегрирования методом Монте-Карло со стратификацией

n	params	estimate	abs_error
100	pdf=p1	39.1212	0.121152
100	pdf=p2	36.4457	2.55427
100	pdf=p3	33.3108	5.68919
1000	pdf=p1	39.2862	0.286226
1000	pdf=p2	37.9541	1.04594
1000	pdf=p3	35.8112	3.18884
10000	pdf=p1	39.2081	0.208145
10000	pdf=p2	38.5303	0.469699
10000	pdf=p3	41.5053	2.50527
100000	pdf=p1	39.1087	0.108698
100000	pdf=p2	39.2299	0.22988
100000	pdf=p3	41.0819	2.08193

Результаты интегрирования методом Монте-Карло с выборкой по значимости

n	params	estimate	abs_error
100	pdfs=p1,p3; weights=balance	38.0487	0.95125
100	pdfs=p1,p3; weights=power	39.761	0.761
1000	pdfs=p1,p3; weights=balance	39.3245	0.324511
1000	pdfs=p1,p3; weights=power	38.7395	0.260474
10000	pdfs=p1,p3; weights=balance	39.031	0.031005
10000	pdfs=p1,p3; weights=power	38.9079	0.092087
100000	pdfs=p1,p3; weights=balance	38.9982	0.001798
100000	pdfs=p1,p3; weights=power	39.0416	0.041572

Результаты интегрирования методом Монте-Карло с многократной выборкой по значимости

n	params	estimate	abs_error
100	pdf=p1, r=0.5	35.1301	3.86986
100	pdf=p1, r=0.75	40.0651	1.06508
100	pdf=p1, r=0.95	40.6088	1.60881
100	pdf=p2, r=0.5	34.7754	4.22464
100	pdf=p2, r=0.75	35.437	3.56304
100	pdf=p2, r=0.95	39.4239	0.42386
100	pdf=p3, r=0.5	25.767	13.233
100	pdf=p3, r=0.75	32.3557	6.64426
100	pdf=p3, r=0.95	66.9072	27.9072
1000	pdf=p1, r=0.5	37.1673	1.83275
1000	pdf=p1, r=0.75	38.9946	0.00539
1000	pdf=p1, r=0.95	38.4755	0.524486
1000	pdf=p2, r=0.5	36.0101	2.98994
1000	pdf=p2, r=0.75	39.4858	0.485758
1000	pdf=p2, r=0.95	36.9543	2.04568
1000	pdf=p3, r=0.5	35.5487	3.45129
1000	pdf=p3, r=0.75	34.6193	4.3807

Результаты интегрирования методом Монте-Карло с использованием русской рулетки (1/2)

1000	pdf=p3, r=0.95	36.158	2.84198
10000	pdf=p1, r=0.5	39.0919	0.091851
10000	pdf=p1, r=0.75	39.3635	0.363492
10000	pdf=p1, r=0.95	39.0307	0.030746
10000	pdf=p2, r=0.5	38.4716	0.528392
10000	pdf=p2, r=0.75	40.105	1.10498
10000	pdf=p2, r=0.95	38.3327	0.667296
10000	pdf=p3, r=0.5	40.5888	1.58884
10000	pdf=p3, r=0.75	41.666	2.66598
10000	pdf=p3, r=0.95	37.1699	1.83011
100000	pdf=p1, r=0.5	39.2011	0.201145
100000	pdf=p1, r=0.75	39.1565	0.156525
100000	pdf=p1, r=0.95	39.0677	0.067711
100000	pdf=p2, r=0.5	38.7119	0.288125
100000	pdf=p2, r=0.75	38.8873	0.112735
100000	pdf=p2, r=0.95	38.7401	0.259921
100000	pdf=p3, r=0.5	43.9835	4.98353
100000	pdf=p3, r=0.75	38.0964	0.90357
100000	pdf=p3, r=0.95	38.7402	0.259778

Результаты интегрирования методом Монте-Карло с использованием русской рулетки (2/2)

method	n	params	estimate	abs_error
simple	10000	-	38.9943	0.005676
stratified	10000	step=1	39.0022	0.002176
importance	10000	pdf=p1	38.9897	0.01033
multi_importance	100000	pdfs=p1,p3; weights=balance	38.9982	0.001798
russian_roulette	100000	pdf=p1, r=0.95	38.9234	0.07663

Лучшие результаты от каждого метода

Вывод

В ходе данной лабораторной работы я научился численно интегрировать функцию с помощью метода Монте-Карло и нескольких его вариаций.